

Hidden Lake Anonymous Network

Kovalenko Gennady Alexandrovich (@number571)

June 8, 2026

Annotation. The Hidden Lake network, being a QB network by nature, also represents a number of new architectural solutions not previously used in anonymous systems. Based on the principle of microservice architecture, such a network can not only add but also remove functions as needed, without changing the overall operating mechanism. Based on blind routing and full message encryption, such a network connects all nodes in the system, preventing long-term monitoring of connections and communication. Understanding the general principles of the network's operation based on its mathematical models can provide not only an assessment of the correct functioning of the entire system but also a possible direction for the future development of anonymous communications.

Keywords: hidden systems; anonymous networks; decentralized networks; theoretically provable anonymity; qb-problem; microservice architecture; gp/12 protocol stack; hidden lake network; post-quantum cryptography;

Content

1. Introduction.....	2
2. QB task.....	2
2.1 Formal proof.....	8
2.2. Disadvantages of QB networks.....	9
2.3. Active observations.....	11
2.4 Comparison with other tasks.....	13
3. Encryption function.....	15
3.1. First stage of encryption (hybrid scheme).....	15
3.2. First stage of encryption (symmetric scheme).....	16
3.3. Second stage of encryption.....	17
4. Networking.....	18
4.1 Microservice Architecture.....	18
4.2. GP/12 Protocol Stack.....	21
5. Software implementation.....	23
5.1 Structural parameters.....	23
5.2 Configuration parameters.....	27
6. Conclusion.....	28
6.1. Applied application.....	29

1.Introduction

The Hidden Lake (HL) anonymity network is a decentralized F2F (friend-to-friend) [1] anonymity network with theoretical provability [2, p. 49]. Unlike well-known anonymity networks such as Tor, I2P, Mixminion, Crowds, and others, the HL network is capable of resisting global observer attacks. The Hidden Lake network does not rely on criteria such as 1) the level of network centralization, 2) the number of nodes, 3) the location of nodes, and 4) the connections between nodes in the network to anonymize its traffic, making the system abstract [2, p. 144].

2. QB task

The Queue Based (QB) problem [2 p.149] is the core of the Hidden Lake anonymity network, which enables theoretically provable anonymity. QB networks are one of the simplest anonymization problems in terms of software implementation.¹, in comparison with other representatives of theoretical provability in the form of DC (Dining Cryptographers) [3, p.225] and EI (Entropy Increase) [2, p.165] networks.

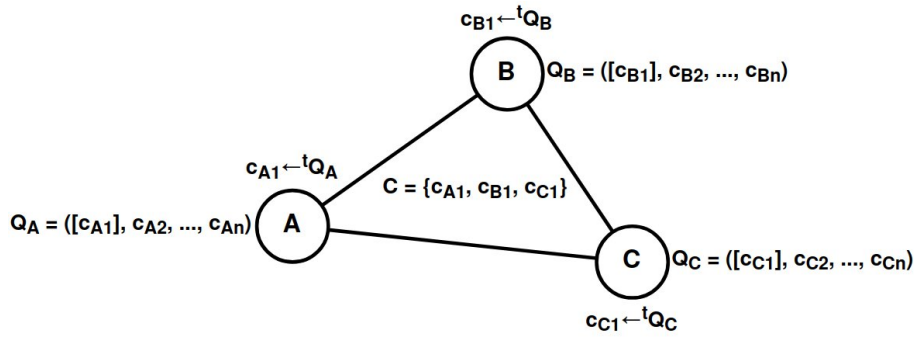


Figure 1. QB network with three participants A, B, C

Definition 1. The task of anonymization should be understood as a cryptographic protocol, the purpose of which is to hide the true relationship between $(s \in S, r \in R)$ from $o \in O$ in the system $\Sigma(S, R, O)$, where S is the set of senders, R is the set of receivers, and O is the set of observers. In special cases, an observer $o \in O$ can be represented by one of the subscribers of direct communication, i.e., $o \stackrel{?}{=} s, o \stackrel{?}{=} r$.

In formal language, a QB network can be described as a system of the following type:

$$QB-net = \Sigma^{ni=1}(T = \{t_i\}, K = \{k_i\}, C = \{(c \in \{E_{kj}(m), E_{ri}(v)\}) \leftarrow {}^i Q_i\})$$

Where n is the number of nodes in the system, K is the set of encryption keys, T is the set of generation periods, C is the set of encrypted messages, Q is the queue of encrypted messages, i, j are the identifiers of individual nodes, E is the encryption function, m is the plaintext message, v is the false message, r is the encryption key not in the set K .

The above system can be represented by four states:

¹ The MA anonymity network is written in 100 lines of Go code, using only the standard library. Repository: <https://github.com/number571/micro-anon>.

1. $Q_i \leftarrow (c = Ek_j(m))$, where $k_j \in K$, $c \in C$. The plaintext message m is encrypted with the recipient's key $k_j \in K$. The encryption result $c = Ek_j(m)$ is placed in the queue Q_i ,

2. $Q_i \leftarrow (c = Er_i(v))$, if $Q_i = \emptyset$, where $r_i \notin K$, $c \in C$. The false message v is encrypted with the recipient-less key $r_i \notin K$. The encryption result $c = Er_i(v)$ is placed in the Q_i queue,

3. $c \leftarrow t Q_i$, where $t \in T$, $c \in C$. At each time period t , an encrypted message c is taken from the queue $Q_i \in \{Ek_j(m), Er_i(v)\}$ and is sent to all network participants,

4. $m' = Dk_{i-1}(c)$, where $c \in C$. Each participant attempts to decrypt the encrypted message it received with its key k_{i-1} . If the ciphertext cannot be deciphered, $m' \neq m$, then this means that the recipient is either someone else (the key $k_j \in K$ was used), or no one (the key $r_j \notin K$ was used).

Lemma 1 [L1]. For any element $r \in R$, with ciphertext $c = Er(v)$, there always exists an r^{-1} that leads to the decryption of the false message $v = Dr^{-1}(c)$.

Proof L1. The set R is defined by the difference of two sets $U \setminus K$, when $U = K \cup R$, which is the set of keys of the encryption function. For any $u \in U$ is mapped into the set of ciphertexts: $\bigcup_{u \in U} u \rightarrow C$, for which there will also exist a u^{-1} that performs the inverse mapping to the set of plaintexts: $\bigcup_{u \in U} u^{-1} \rightarrow M$, based on the fact that the system $\Sigma(M, C, U, E, D)$ is a cipher [4, p.75].

The anonymity of QB networks is based on the difficulty of determining the state of the encrypted message c , namely what it is: $Ek(m)$ or $Er(v)$. The order of messages Q , in turn, guarantees that there will always be a message c generated by the system in a time period equal to t , regardless of the nature of the message itself. In the absence of genuine messages, queue Q can be viewed as a queue of exclusively false messages $Er(v)$. When the true message appears $Ek(m)$, this begins to replace the false one $Er(v)$ in a certain period of time t . As a result, the QB network becomes a noise generator with the function of briefly replacing random traffic with valid traffic, and the indistinguishability of encrypted messages from each other becomes a key factor in anonymity.

Lemma 2 [L2]. If there are two encryption keys k, r in the system (generating true and false ciphertexts, respectively), determining the truth of the selected ciphertext c_i from the finite set $C = \{c_1, c_2, \dots, c_n\}$ is reduced to the problem of indistinguishability of ciphertexts from plaintexts and encryption keys.

Proof L2. The problem of determining the truth of a ciphertext c_i from a set C is reduced to establishing its belonging to one of two states: $Ek(m_i)$ or $Er(v_i)$. In this concept, the encryption function E must have the property of ciphertext indistinguishability: 1) by the plaintexts m_i, v_i ; 2) by the encryption keys used.² k, r The first property guarantees that even if the observer knows the

² Some asymmetric encryption algorithms lack a guarantee of ciphertext indistinguishability based on encryption keys, such as RSA. A public key in RSA is represented by two values $\{e, N\}$, where e is the public exponent (usually the constant 216+1) and N is the modulus, represented by the product of two prime numbers. Suppose there are two public keys $pub1 = \{e, N1\}$ and $pub2 = \{e, N2\}$. One of the two moduli will always be greater than the other, i.e. $N1 > N2$ (otherwise, the keys are the same). As a result, if an observer receives a ciphertext $c = m \cdot e \cdot N$ and $c > N2$, this means that the ciphertext c was created based

texts or patterns of true/false messages, associating them with the encryption result (ciphertext) is computationally infeasible. The second property guarantees that even if the observer knows the public encryption keys, associating them with the encryption result is also computationally infeasible. As a result, when using a secure function E and parameters $k \in K, r \in R$, the upper bound of the search begins to be determined by the number of iterations of the complete search equal to $|K \cup R| = |U| = |\{u_1, u_2, \dots, u_{|U|}\}| \rightarrow Du(c)$, at $K \cap R = \emptyset$ accordingly, due to the fact that all the values of u_i being iterated belong to the general cipher system (Lemma 1).

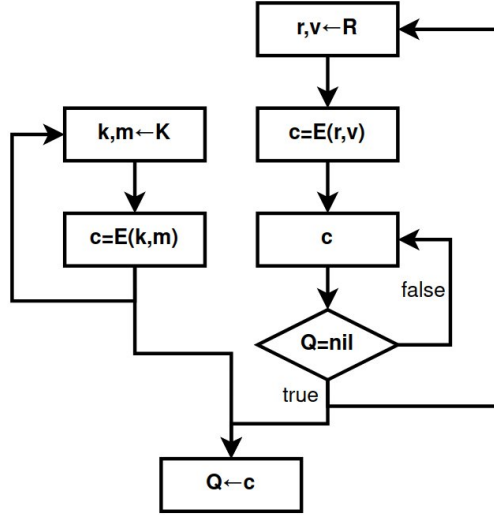


Figure 2. Algorithm for filling a queue in a QB task, Where R is a false message generator, K is a true message generator

Thus, the anonymity of QB networks is determined not only by the disconnect between the sender and receiver for the global observer, but also by the lack of connection in the very act of sending and receiving information. In other words, passive observers, including the global observer, face the impossible task of determining the state of the subject, namely:

1. Does participant i send a true message $E_{k_j}(m)$ in period t_i ??
2. Does participant i receive any message $D_{k_{i-1}}(c)$ in periods equal to $T \setminus \{t_i\}$??
3. Is participant $i \rightarrow E_r(v)$ inactive in the analyzed period t_i ??

In all such scenarios, without being one of the nodes directly participating in the communication and without exerting any influence on the queue Q_i of the analyzed participant i , i.e. without being a node exerting active observation, the task is considered infeasible if the encryption algorithm E and the keys k, r are reliable.

Definition 2. The task of anonymization is considered theoretical³, if the presence of any passive observer $\forall o \in O$ in the system $\Sigma(S, R, O)$, including the global one, in no way affects the quality of hiding the true connection between $(s \in S, r \in R)$.

Algorithm 1. Participant functioning in the QB-net system in pseudocode language

INPUT: queue Q, period t, functions E, D, encryption keys k, r

OUTPUT: a subset of ciphertexts $X \in C$

thread-1. (*Generating true ciphertexts*)

while (true) {

$k_j \leftarrow \text{INPUT_STREAM} (*\text{Input receiver key}*)$

$m \leftarrow \text{INPUT_STREAM} (*\text{Plaintext Input}*)$

$Q \leftarrow (c = E_{k_j}(m))$

}

thread-2. (*Generating false ciphertexts*)

while (true) {

$v \leftarrow \text{RANDOM_STREAM} (*\text{Receiving fake text from the KSGPCH}*)$

$c = E_r(v)$

 while ($Q \neq \emptyset$) {} (*Waiting for an empty queue*)

$Q \leftarrow c$

}

thread-3. (* Sending ciphertexts to the network *)

while (true) {

 sleep(t) (* Wait for period *)

$c \leftarrow Q$, where $c \in X$

 QB-net $\leftarrow c$ (* Writing ciphertext to the network *)

}

thread-4. (* Accepting ciphertexts from the network *)

while (true) {

$c \leftarrow \text{QB-net}$, where $c \in C \setminus X$ (*Reading ciphertext from the network*)

 if valid($m = D_{k^{-1}}(c)$) { (* Checking the correctness of the decryption *)

 OUTPUT_STREAM $\leftarrow m$ (* Output of the received plaintext *)

 }

}

QB networks also have a number of interesting and not entirely obvious features. For example, the period t_i of each individual participant i does not necessarily have to be a constant value. The period can vary over time or even be random. This behavior will not affect the anonymity of the nodes as long as the fact of ciphertext deferredness c , represented by the message queue Q , exists. If messages can be sent without this queue, then the system will cease to be theoretically provable..

³One of the possible ways to achieve theoretical provability is to reduce the problem of hiding a connection $(s \in S, r \in R)$ to the problem of hiding the source of this connection $s \in S$. Based on this approach, DC, EI, and QB networks are formed.

Thus, unlike DC networks, where the period T is represented by only one common value $T = \{t\}$, QB networks make the period not only subjectively (individually) configurable $T = \{t_1, t_2, \dots, t_n\}$, but also not necessarily static for each generated message $t \in [l:k]$, where $l \leq k$. This property allows QB networks to avoid cooperating with individual nodes during the period, and also to more effectively hide the pattern of user affiliation with the anonymizing traffic.

Theorem 1 [T1]. The theoretically provable anonymity of the QB-net system is based on the systematic generation of the set of ciphertexts $C = C_r \cup C_k = \{c_1, c_2, \dots, c_n\}$ and on the indistinguishability of its subsets with respect to true C_k and false C_r ciphertexts.

Proof T1. Let $C_r = \{c_{1r}, c_{2r}, \dots, c_{mr}\}$ be the set of false ciphertexts, and $C_k = \{c_{1k}, c_{2k}, \dots, c_{nk}\}$, on the contrary, be the set of true ciphertexts for $m, n \geq 0$ accordingly, then when they are combined, the set of all ciphertexts is created: $C_r \cup C_k = \{c_{1r}, c_{2r}, \dots, c_{mr}, c_{1k}, c_{2k}, \dots, c_{nk}\} = \{c_1, c_2, \dots, c_{m+n}\} = C$. Let the deanonymization problem $P(d)$ be understood as the unambiguous determination of the existence or absence of a true message ($n > 0$?) in a set of ciphertexts C . The deanonymization problem, in turn, relies on two other problems: indistinguishability $P(i)$ and systematicity $P(s)$. If one of the two subproblems is solved, the deanonymization problem will also be considered feasible, which can be expressed in disjunctive form: $P(d) = P(i) \vee P(s)$.

Indistinguishability problem $P(i)$ can be determined by two possible situations: 1) either by the method of finding the true ciphertext $c_i = Ek(m_i) \in C_k$; 2) or, on the contrary, by means of proving the absence of true ciphertexts $c_i \notin C_k = \emptyset$. *The problem becomes trivial in the absence of ciphertexts in general, since $C = \emptyset \rightarrow C_k = \emptyset$. If $C \neq \emptyset$, then the problem is reduced to the problem of the ratio of ciphertexts $c_i \in C$ to their original subsets: C_r or C_k , which, as shown earlier, is a computationally difficult problem (Lemma 2).*

Systematicity problem $P(s)$ can be determined by finding additional connections in the ciphertext generation mechanism $C = \{c_1, c_2, \dots, c_n\}$. If we take a special case $C = \emptyset$, then the problem becomes infeasible, since the coherence of the ciphertexts will be a priori absent. Further, let $|C| = 1$, i.e. $C = \{c\}$, then the systematicity problem will be reduced to the question: "as a result of which event x was the ciphertext c obtained?" If the event x has no connection with any plaintext m (received, being received, sent, or being sent), i.e. the ciphertext c was not created as a result of the appearance of m as an event, then x should be considered as an independent event. If we assume that there is some algorithm A that generates ciphertexts c_i through an independent event x , i.e. $c_i \leftarrow A(x)$, where $c_i \in C$, and at the same time $C = C_r$, then in the generation mechanism there will be a priori no additional connections other than the main connection represented by the event x , since $C_k = \emptyset \rightarrow c_i \notin C_k$. Now, if we assume the opposite: $C = C_r \cup C_k$, where $C_k \neq \emptyset$, then the preservation of a single generation connection, in the face of an independent event x , becomes possible if and only if the ciphertexts $c_i \in C_k$ will be created together and as a result of the same algorithm A as the ciphertexts of the subset C_r , by fulfilling the condition: $(c = Ek(m))$ if $(m \neq null)$ else $(c = Er(v)) \leftarrow A(x)$. As a result, the true relationship ($m \neq null$) is encapsulated in an independent connection x , and thus does not reveal its generating nature.

Investigation T1:1. A finite set of ciphertexts C can be viewed as the result of the step-by-step generation of subsets $C_1 = \{c_1\}$, $C_2 = \{c_1, c_2\}$, ..., $C_n = \{c_1, c_2, \dots, c_n\}$ and as the completion of network communication as a whole. Thus, if the conditions of indistinguishability and

systematicity are satisfied by the set $C = C_n$, the subsets $C_i \subseteq C_n$ continue to inherit their execution equally.

Investigation T1:2. Let t be the period of generation of generated subsets C_i . In this case, t is also input condition and independent event for the ciphertext generation algorithm: $c_i \leftarrow A(t)$. Since the period t is based on algorithm A and the generation of ciphertexts is not related to plaintexts, then its variable characteristic in the form of staticity ($t = T$) or dynamism ($t \in [l; k]$) is not able to influence the quality of anonymity.

Investigation T1:3. Independent events $x_1, x_2, \dots, x_n$, associated with participants $1, 2, \dots, n$, do not necessarily have to be represented by a common constant/static value $= x$, since these events belong equally to the common class of independent events X . As a result, the existence of an inequality x_i is allowed $\neq x_j$, for any i, j values.

Example T1:1. In a QB problem, algorithm A is understood as Q —a "queue" structure, and an independent event x is most often understood as the parameter $x=t$ —the generation period. Although t is a simple condition for the formation of independence, it is not the only one. For example, an independent event $x=n$ could be understood as the formation of a ciphertext: $c \leftarrow A(n)$ based on the n -th number of ciphertexts received by the system from other participants. The lack of dependence on plaintexts naturally leads to similar theoretical provability and indicates the possibility of the system using several different events.

Example T1:2. Sending all ciphertexts $C = \{c_1, c_2, \dots, c_n\}$ to the network at once is also an independent event $x=a: C \leftarrow A(a)$, albeit specific, since 1) it excludes any interactivity in the form of requests and responses; 2) it has no information about the nature of the generation of the set C itself. This example is interesting from a theoretical point of view in that, with some probability, all ciphertexts in the set C must necessarily remain false. Otherwise, two conditions will be violated at once: 1) indistinguishability, where the inequality C_k will become known $\neq \emptyset$; 2) systematicity, where event a will depend on the open texts: $A(a)=A(m)$.

Example T1:3. A slight difference in the algorithm A when generating true and false ciphertexts can disrupt the systematicity of generation. Suppose that a static period t is specified as a condition for two algorithms P, Q , and that the ciphertext generation procedure itself takes a long time r , such that $0 < r < t$. Further, if we assume that false ciphertexts are generated immediately at the moment of sending ($c_i = Er(v_i) \leftarrow P(t)$), then the ciphertext c_i will be sent after a time equal to $t + r$. In turn, if genuine messages are generated before the moment of sending, i.e. first placed in the queue ($c_i = Ek(m_i) \leftarrow Q(t)$), and only then are they sent, then the time of their sending will be specified by the constant t at the next stage of generation. Thus, due to the use of different generation algorithms, P and Q , respectively, the systematic generation of ciphertexts c_i is violated due to the difference r created, although a common, independent event t is used for the two algorithms.

Example T1:4. Difference between algorithms $P(t)$ and $Q(t)$ is determined only by the absence or existence of a message queue. Theoretically, both algorithms are based on an independent event t , but in practice they differ in that, $P(t)$ The generation of true ciphertexts will be more difficult, since it requires their manual creation at a specific time period t due to the lack of a mechanism for

deferring messages until a specified condition. Algorithm with a queue $Q(t)$, on the contrary, allows generating and storing messages at any point in time, regardless of the specified period t .

Furthermore, QB networks assume the use of asymmetric cryptography by default, and as a consequence, keys $k_i \neq k_{i-1}$ are not directly related to each other. However, QB networks are quite capable of relying solely on symmetric cryptography, where $k_i = k_{i-1}$. In this case, the keys will represent not each individual participant in the system out of n possible ones, but rather the direct relationship between its participants from $n(n - 1)/2$ possible edges of the graph (system), which also leads to the emergence of shared keys of the form $k_i = k_j$ for some i, j participants. If the system has a routing mechanism, then for decryption the recipient will have to use not just one specific key k_{i-1} , but all known keys $k_1, k_2, \dots, k_n$ via a brute-force method.

The use of symmetric cryptography can increase the cryptographic strength of the QB-net system in preparation for post-quantum cryptography or as a result of its onset, since it is known that modern symmetric algorithms with a large key length (256 bits or more) are quantum-resistant [5, p. 131], while newly developed asymmetric algorithms, from a conservative point of view, may be too new to be considered secure, including even for classical computers.

Furthermore, the use of symmetric algorithms can make the QB-net system not only quantum-resistant, but also absolutely cryptographically secure if the Vernam cipher is used as the symmetric algorithm [3, p. 65]. In this case, it will be necessary to calculate the maximum possible amount of gamma G used for the required time interval P (the number of generation periods) given the number of interlocutors N (including the false recipient) and the message length L .

$$G = P \times (1 + N) \times L$$

However, the use of exclusively symmetric cryptography also complicates practical use and brings with it a number of additional costs:

1. The general system of quantitative storage and distribution of keys becomes more complex: $2n$ in asymmetric cryptography, $n(n - 1)/2$ in symmetric cryptography [3, p.278],
2. Decrypting the data will involve a linear search of all known symmetric keys k_i , which in the presence of asymmetric keys required only one key k_{i-1} ,
3. The threat model will be reduced from the ability of an active intermediary to replace public keys to the ability of a passive intermediary to view secret keys [3, p.80],
4. Imitation inserts, unlike digital signatures of the asymmetric section of cryptography, do not allow for unambiguous confirmation of the authorship of messages [6, p.46].

2.1 Formal proof

To determine the anonymity quality of the QB-net system, probabilistic assessment, entropy metric, and correlation analysis can be used. The main goal is to prove that for an outside observer, including a global observer, the uncertainty regarding the source or recipient of a message is maximal, leading to the concealment of the very fact of communication.

Threat Model Let the network be represented as a set of nodes: $S = \{s_1, s_2, \dots, s_N\}$, where $|S| = N$. The global observer is an adversary with the ability to view all network traffic. All nodes are

considered potentially compromised except the sender (Sender) and the recipient (Recipient). The threat model is limited exclusively to the network portion of the interaction and does not address the security of the working environment during node startup and/or operation.

Probability assessmentIn a QB-net system, each node broadcasts packets to all neighbors. Assume that all packets are of fixed length L and encrypted using a scheme E with the IND-CPA/IND-CCA2 property.⁴If node A sends a message and node B receives it, the global observer will have the following data:

- *Sender probability*: since all nodes generate packets with a predetermined periodicity Δt (constant noise), the probability that a particular node s_i is the sender of message M is: $P(\text{Sender} = s_i) = 1 / N$,

- *Recipient probability*: Since the packet is encrypted with the recipient's key and sent to everyone, it is externally impossible to determine who exactly was able to decrypt it. The probability that node s_j is the true recipient of message M is: $P(\text{Recipient} = s_j) = 1 / N$.

Entropy indexIf the probability distribution between the participants is uniform and remains so throughout the communication, then the entropy is considered to be maximum: $H(X) = -\sum_{i=1}^N p_i \log_2(p_i) = \log_2(N) = H_{\max}$ At maximum entropy, the degree of anonymity d becomes equal to one: $H(X) / H_{\max} = 1$, which already indicates the presence of theoretically provable anonymity.

The anonymity of the system is considered perfect if the a posteriori probability of identifying the sender s_i remains equal to the a priori, with all perfect analysis of the traffic T by any external observer: $P(\text{Sender} = s_i | T) = P(\text{Sender} = s_i) = 1 / N$. In the QB-net system, this is achieved due to the system parameters (L, E, Q), due to which the traffic vector $T = (C_1, C_2, \dots, C_x)$, at any given moment in time t , has an entropy that is independent of the initiator of the communication: $H(\text{Sender} = s_i | T_t) = H(\text{Sender} = s_j | T_t), \forall s_i, s_j \in S, \forall t \in \{1, 2, \dots, x\}$. Thus, the amount of mutual information between any given sender and the observed traffic remains equal to zero: $I(\text{Sender}; T) = H(\text{Sender}) - H(\text{Sender} | T) = 0$, which also indicates the concealment of the very fact of sending real messages M .

Correlation analysisThe quality of anonymity when sending messages can be measured by the covariance between incoming and outgoing traffic, using the example of a request and response. Since the QB-net system uses a queue with a generation period of Δt , the transfer function F will look like: $F(\text{Input}) \rightarrow \text{Output} + \Delta$, Where $\Delta = -t \bmod \Delta t$, t is the departure time. Since outgoing traffic *Output* is solely the result of queue Q and can be generated in the absence of *Input* as such, i.e. $P(\text{Output} | \text{Input}) = P(\text{Output})$, the correlation of events becomes zero: $\text{Cov}(\text{Input}, \text{Output}) = 0$, indicating the impossibility of timing attacks.

2.2. Disadvantages of QB networks

Unfortunately, QB networks are not ideal and also have problems and shortcomings inherent to their class, some of which lead to limited practical use, while others lead to network availability issues:

1. **Linear network load**. In QB networks, everyone sends messages to everyone, with the sole purpose of making it impossible to narrow the scope of actual communication between system

⁴ IND-CPA— indistinguishability for chosen-plaintext attacks, IND-CCA2 — indistinguishability for adaptive chosen-ciphertext attacks.

participants. Blind (flood) routing becomes the routing algorithm.[7, p.398], as a result of which the increase in the number of nodes has a linear $O(n)$ effect on the increase in the load of the entire system,

2. Order-locked. Each node in a QB network is tied in some way to its own message order Q , where every time period equal to t , an encrypted message is sent to the network. This means that increasing node throughput is only possible in three scenarios, each of which will increase the load on the entire network.:

1. Increase the size of transmitted open messages $m_1, m_2, \dots, m_n$,
2. Increase the number of ciphertexts sent at a time $c_1, c_2, \dots, c_n \leftarrow t Q$,
3. Reduce the message generation period t ,

3. Connectivity of communication participants. QB networks do not assume anonymity between nodes directly involved in communication. This is primarily due to the fact that QB networks lack the concept of information polymorphism [2, p. 62], that is, the state of information in a system in which its appearance constantly changes from node to node, for both internal and external observers. This property allows the connection between sender and receiver to be broken through the transmitted object, i.e., the information itself.

Due to all the above mentioned disadvantages, the scope of application of QB networks becomes more limited:

1. Due to the linear network load and queue-based nature of QB networks, they scale poorly and can only operate in small groups of up to N participants. The maximum number of participants is limited by the network's bandwidth, as well as the capacity of the nodes constantly encrypting and decrypting outgoing/incoming traffic. Because of this drawback, implementing streaming services and video/audio calls becomes either very difficult or completely impossible.

2. Due to the interconnectedness of communication participants, the range of applications where anonymity between nodes is important is limited. Consequently, the most relevant combination of QB networks with F2F (friend-to-friend) networks has emerged, where the communication link is established by two system participants, rather than just one. This does not solve the problem of the lack of anonymity between the connected nodes, but it does provide additional protection against uncoordinated automatic linking and a more explicit trust relationship, assuming that neither participant will attempt to de-anonymize the other.

Algorithm 2.Message filtering in F2F networks in pseudocode

INPUT: set of friends F , sender s , handler function h , message m
OUTPUT: processed message $h(m)$ OR completion of the algorithm
if ($s \notin F$) {
 return (End of algorithm *)*
}
return $h(m)$ (Message processing *)*

2.3. Active observations

Based on the above, it was previously proven that the QB problem is immune to any passive observations. This suggests that this problem belongs to the class of anonymization problems with theoretical provability. In turn, theoretical provability is not absolute, as it boils down to solving only the problems of passive observations, ignoring and bypassing active ones. As will be shown below, the QB problem does not have absolute anonymity and may be vulnerable to specific active observations.

Suppose that a QB network has a relay role that hides subscribers' network addresses (IP addresses) from each other by redirecting traffic, and that one of the subscribers cooperates with a global observer. In this case, the $IP \leftrightarrow ki$ mapping problem becomes trivial, since a subscriber only needs to receive one true message $m = Dk^{-1}(c)$ from its interlocutor. The global observer can then determine its first occurrence based on the received ciphertext $c = Ek(m)$, thereby deanonymizing the sender or recipient (their network location). This indicates a strong connection between the communication subscribers and, as a consequence, a lack of anonymity towards each other.

The situation can be complicated for observers by adding channel encryption between nodes, as done in Crowds [8]. In this case, a global observer will not be able to explicitly link a sent and received message, because it will constantly change its form as it passes from one node to another:

$$Ek_3(m) \rightarrow Ek_2(m) \rightarrow Ek_1(m) \rightarrow m$$

In any case, this property poorly serves the function of distinguishing nodes from each other regarding routing information m . Consequently, the global observer's task becomes the implantation of controlled nodes into the system next to every other node. In this scenario, the observer can again easily solve the $IP \leftrightarrow ki$ problem.

Definition 3. The task of anonymization is considered absolute⁵, if the presence of any active observer $\forall o \in O$ in the system $\Sigma(S, R, O)$ does not in any way affect the quality of hiding the true connection between $(s \in S, r \in R)$.

Another way to solve the problem is to make the information polymorphic. Polymorphism in anonymous networks is most often achieved through multiple encryption, where layers of encryption are gradually removed during transmission from one node to another, as in Tor [9], I2P [10], and Mixminion [11]. This property allows for the separation of subscribers' communications, thereby anonymizing them:

$$Ek_3(Ek_2(Ek_1(m))) \rightarrow Ek_2(Ek_1(m)) \rightarrow Ek_1(m) \rightarrow m$$

Technically, multiple encryption could be implemented in a QB network, i.e., making information polymorphic, to further differentiate communication participants from each other and, as a result, prevent linking attacks. However, in this case:

⁵Currently, only the DC problem, in its classical definition, can ensure absolute provability. Disabling or slowing down one participant, by an external active observer, will lead to a blockade of the entire network. Sending a large number of packets will only lead to collisions and accompanying DoS/DDoS attacks, but will not reveal the source of the message. Sending multiple requests from an internal active observer to one of the system participants, followed by analysis of the received responses, similarly leads to nothing.

1. The speed of information transfer will decrease, since each routing node will have to store the message it received earlier in turn.,
2. The anonymization system as a whole will become more complex, since instead of a single anonymization task = QB, a composition of tasks = QB + Onion will be used,
3. The composition of tasks = QB + Onion has a number of subtleties with more complex active observations [2, p.159], but still deanonymizes network subscribers.

Theorem 2 [T2]. If there is an active internal observer f_i in the QB-net system, in the role of the interlocutor of node i , the task of anonymization will always be reduced to concealing the connection between these communication subscribers.

Proof T2. Let's assume that the QB problem can be defined by the fact of communication obfuscation in the presence of an active internal observer f_i , playing the role of interlocutor of node i . In this case, we arrive at a contradiction, because attacker f_i , when sending or receiving messages from interlocutor i , will a priori know that during specific time intervals $t_1, t_2, \dots, t_n$, there was an exchange of plaintexts $m_1, m_2, \dots, m_n$, and therefore the very fact of communication obfuscation of interlocutor i was violated. As a result, the deanonymization problem begins to be reduced to finding a specific connection.⁶, and not to the presence of the fact of the existence of this connection.

Further, if we assume an attack scenario on a QB network in which, in the circle of friends $F = \{f_1, f_2, \dots, f_n\}$ of participant i , there exists an attacker f_j in the role of an active observer capable of sending requests and receiving responses from i , then the attack model will be reduced to an analysis of the state of the queue Q_i . Let us further assume that participant i has set a static message generation period equal to t_i . In this case, f_j will be able to send messages at certain time intervals $\{t'_i, 2t'_i, \dots, nt'_i\}$, dependent on the time period $t_i \Rightarrow (kt'_i = kt_i + x)$, where $x \in [0; t_i)$, send a request to Rkt_i participant i for the purpose of analyzing the response time. If the response received after the Rkt_i request, will be generated in the range dt_i , where $d > 1$, then this will indicate the fact of real communication between participant i and anyone else in the network in the set of periods $D = \{(1+k)t_i, (2+k)t_i, \dots, (d-1+k)t_i\}$, since more than one period was required for the response. If $d = 1$, then participant i did not cooperate with anyone in the period $(1+k)t_i$.

Thus, the attack described above reduces the anonymity of QB networks from concealing the fact of activity to concealing the communication links between subscribers. In other words, with such active surveillance, it is now possible to determine the state of a subject, whether they are sending or receiving legitimate messages, but the following issues remain unclear:

1. Which nodes did the eavesdropped participant communicate with? in the set of periods D ?
2. Was the listened participant i the initiator of the requests when set D ?
3. Can participant i intentionally generate false messages as true ones?

⁶This statement does not extend to other anonymization problems with theoretical provability. For example, even if there is a relation i in the DC problem $\leftrightarrow f_i$, it will not lead to a reduction in the threat model from concealing the initiator of the communication to concealing the communication between the parties. This is due to the fact that the DC task involves information polymorphism, which prevents observers, including the interlocutor, from unambiguously determining the originating point.

Further, if we assume a situation in which each node $i \in \{1, 2, \dots, n\}$, who is not an observer, will have at least one active observer f_i among his friends $\in \{f_1, f_2, \dots, f_n\}$, then the first problem can be solved trivially, provided that the attackers will be in cooperation, i.e. $\forall i, j, f_i = f_j$, and participants will use the communication type "request-response". In this case, the queue is almost simultaneously loaded $(d+x)ti$, where $d > 1, x \in \{0, 1, 2\}$ several network participants i will indicate a limitation of the initial observation set due to the division of nodes i, j by filled $Q_i \neq \emptyset$ and empty $Q_j = \emptyset$ queues.

The solution to the second problem can be based on the conditions of the first, when there will be a set of active and cooperating observers $f_i \in \{f_1, f_2, \dots, f_n\}$, and the participants will use the request-response communication type. In this case, to find the answer, observers will need to detect the fact that communication has begun for all nodes $i \in \{1, 2, \dots, n\}$ by transitioning the queue state from empty to full: $Q_i = \emptyset \rightarrow Q_i \neq \emptyset$. The first node to make such a transition is more likely to become the request initiator.

The solution to the third problem is the most labor-intensive from the point of view of observers, since, firstly, it is associated with solving the problem of indistinguishability of ciphertexts $P(i)$, which is a computationally difficult problem, and secondly, the absence of a solution to the third problem complicates the identification of patterns in the first two problems by hiding the exact state of the queues of subscribers of the communication " $Q_i = \emptyset$ ". Although this problem is related to the indistinguishability problem, it is not identical to it, as it also involves the systematicity problem $P(s)$. Therefore, to prove the security of the third problem, its complexity must be reduced to the deanonymization problem $P(d)$. However, this is impossible, since the response to the observer itself becomes a procedure for violating the independence of event x in the systematicity problem. Due to this, observers can be confident that at the moment of response dt_i , participant i could not simultaneously be responding to another subscriber. Thus, the third problem cannot guarantee anonymity and, with longer observation, may still exhibit patterns characteristic of the solution to the first problem.

The problem can be solved by increasing the number of queues $Q \rightarrow \{Q_1, Q_2, \dots, Q_n\}$ on one node with their connection to communication subscribers $i \in \{1, 2, \dots, m\}$. In this case, the number of queues must be a priori set to a static value n , so that it is impossible to determine the number of linked friends m , i.e., the number of friends m must always have the comparison: $m \leq n$. As a result, the expansion of queues will lead either to increasing the number of ciphertexts sent at a time: $c_1, c_2, \dots, c_n \leftarrow t Q_1, Q_2, \dots, Q_n$, or increasing the ciphertext generation period: $c_i \leftarrow nt Q_i$. This, in turn, will serve as a guarantee/proof that an active observer will not be able to influence the message queue of other subscribers, since all of its actions will be bound and limited to a single, disjoint queue.

2.4 Comparison with other tasks

	QB	EI	DC	Onion	Proxy
Absolute provability	-	-	+	-	-
Theoretical provability	+	+	+	-	-
The cumulative effect of anonymity	-	+	-	-	-
Information polymorphism	-	+	+	+	-
Probabilistic routing	-	+	-	+/-	+/-
Frequency of message generation	+/-	-	+	+/-	+/-

Independence of anonymity from connections	+	-	-	-	-
Easy to scale	-	-	-	+	+
Simplicity of software implementation⁷	+	-	-	+/-	+
Anonymity stage	5 [^]	6	1 [^]	4 or 6	3
Representative network⁸	Hidden Lake	-	Herbivore	Tor	Crowds

Table 1.Comparison of anonymization tasks

In terms of its characteristics, the QB problem is closest to the DC problem due to the following features: theoretically provable anonymity, periodic message generation, belonging to the second vector of anonymous communications development [2, p. 71], and difficulty scaling. From a positive perspective, QB differs from DC networks in the following ways: the periodicity of generation can be dynamic, anonymity does not depend on established connections with other participants, and a simpler software implementation. Negative differences are determined by the lack of information polymorphism and the absence of absolute provability. A more detailed and general comparison of the QB problem with other anonymization problems, both theoretical and practical, is presented in Table 1.

Hiding quality	Quality of provability
Hiding communications between subscribers <i>Onion, Proxy</i>	Practical provability <i>Onion, Proxy</i>
Hiding the initiator of communication <i>DC, EI</i>	Theoretical provability <i>QB, EI</i>
Hiding the fact of connection <i>QB</i>	Absolute provability <i>DC</i>

Table 2.Comparison of the qualities of anonymization tasks

The anonymity quality of several problems can be considered in two dimensions: 1) in terms of concealment quality; 2) in terms of provability quality (Table 2). Depending on the chosen method, either the QB or DC problem can be the leading one. For example, the QB problem, although it conceals the very existence of communication, is not absolutely provable. A completely

⁷ Complexity can be measured by the ratio of the number of successful passive/active observations to the number of necessary procedures to prevent such observations. If the number of procedures is small, then the simplicity of the software implementation will be minimal. This, however, does not necessarily reflect the overall security of the network, as a small number of procedures may also indicate a lack of measures to eliminate successful observations. In this case, the simplicity of the software implementation may be a consequence not only of high security but also of a reduced threat model.

⁸A representative network doesn't necessarily have to inherit all the characteristics of the anonymity problem. For example, Herbivore, although a DC network, is neither absolutely provable nor theoretically provable. This is because Herbivore compromises between anonymity and efficiency, thereby weakening its threat model. In contrast, the Hidden Lake network fully adheres to the QB problem concept, even with all its shortcomings. The Crowds network even improves its threat model, inheriting not only the classic Proxy Problem but also adding a probabilistic routing feature.

different dimension is the DC problem, which, while being an absolutely provable problem, does not conceal the existence of the connection.⁹

3. Encryption function

As previously shown, QB networks depend on the quality of the function E and the encryption keys k, r . The quality of the encryption keys is determined primarily by the quality of the RNG (random number generator) and/or the CPRNG (cryptographically secure pseudorandom number generator). The analysis of such generators is complex due to the different environments in which they execute and the means they employ during their execution [6, p. 190]. Therefore, based on the logic of abstraction, we will henceforth assume that the keys are generated in a high-quality and secure manner, thereby focusing exclusively on the logic of the encryption function execution.

$$E(nk, \langle fk \rangle)(m) = E''_{nk}(E' \langle fk \rangle(m))$$

The encryption function in the Hidden Lake network consists of two stages, each with a clearly defined role. The first stage $E' \langle fk \rangle$ comes down to direct and primary encryption of data in order to hide it from unauthorized persons, using either a hybrid encryption scheme $E' \langle fk \rangle = E'(privA, pubB)$ [2, p.125], or symmetrical $E' \langle fk \rangle = E'^k$. The second stage E''_{nk} comes down to dividing the network into several subnetworks by using different encryption keys (network keys).

3.1. First stage of encryption(hybrid scheme)

$$E'(privA, pubB)(m) = (E_{pubB}(k') \parallel Ek'(H(pubA) \parallel s \parallel m' \parallel h \parallel SprivA(h))),$$

$$h = H_{MAC(s)}(pubA \parallel pubB \parallel m'), m' = f(m), k' = [RNG], s = [RNG],$$

Where k' – session encryption key calculated for one message, s – cryptographic salt calculated for one message, m – plain message, $pubA, pubB$ – public keys of participants A, B respectively, $privA$ – private key of participant A, h – hash result, S – signature function, f – function of message complement to constant value, H – hash function, $HMAC$ – function of calculating impersonation insertion based on hash function H . In this scheme, it is assumed that A is the sender of information m , B is the recipient of this information. The security of this function directly depends on the public encryption key $pubB$, which is used to encrypt the subsequent session key k' , from the quality of the RNG / KSGPSN that was generated k' , and also from the security of the encryption functions themselves $E_{pubB}, E_{k'}$.

This scheme is interesting because it hides all information in an encrypted shell, preventing attacks on the identity of the sender or recipient. For example, if the hash value h and signature $SprivA(h)$ were not in the encrypted block Ek' , then it would be possible to attack the analysis of encrypted messages using the existing list of public keys $\{pub1, pub2, ..., pubn\}$,

⁹ The anonymity quality of a given problem can also be considered in terms of the simplicity/ease of transitions between adjacent states. Technically, a DC problem can have communication obfuscation (albeit not ideal) if the probability of collisions is negligible and the encryption itself is end-to-end. At the same time, a QB problem cannot achieve absolute provability, as it cannot withstand a series of active observations. Thus, the anonymity quality of a DC problem can, in total, exceed that of a QB problem.

checking their authenticity $V_{pubi}(SprivA(h)) = h$. Further, if the cryptographic salt s and the hash value h were known, then it would be possible to compile a table of the most frequently occurring messages $\{m_1, m_2, \dots, m_n\}$ with different combinations of participants i, j from the set of all network nodes N using the equality $HMAC(s)(pubi, pubj, f(ml)) = h$.

Moreover, this scheme is self-sufficient [2, p. 121] at the network level of QB networks in the context of flood routing, as it allows for subject identification solely through asymmetric cryptography. Identifying the sender of a message becomes possible through correct decryption, i.e., provided that the recipient of the encrypted message has the required private key.

This stage assumes that the message $f(m)$ has a static value. In other words, each time the encryption function is called, $E'(privA, pubB)$, for all m_i, m_j from $\{m_1, m_2, \dots, m_n\}$ the message length L from the function l is observed, such that $l \Rightarrow l(f(m_i)) = l(f(m_j)) = L$. This is made possible by the preprocessing procedure f , which limits the length of the input message to L and pads the length of the input message to L . The purpose of this procedure is to protect against attacks based on message size analysis, which can reveal the structure of the transmitted message. For example, requests are often smaller in size than responses, transmitted video or audio files are often larger in size than regular text messages, system/automated requests are smaller in size than manually executed ones, etc. [12].

It is also assumed that the recipient of the ciphertext has a list of public keys and their hashes in dictionary format, i.e. $H(pubi) \leftrightarrow pubi$. If decryption is successful, the user receives the transmitted value $H(pubA)$, after which they attempt to match it with the public key in the dictionary. If the public key $pubA$ is not found in the dictionary, the message is ignored. Otherwise, the sender's public key $pubA$, their own public key $pubB$, the cryptographic salt s , and the message m' are used to verify the correctness of the resulting hash sum $h = H_{MAC(s)}(pubA \parallel pubB \parallel m')$. If the sum is incorrect, the message is ignored. Otherwise, using the public key $pubA$ and the received value $SprivA(h)$, the digital signature is verified. If the signature is valid, then message m' is decoded $f^{-1}(m') \rightarrow m$ and is accepted. Otherwise, the message is ignored.

Along with the inherent need to hide information within an encrypted shell, the problem of verifying the ciphertext content arises, as the hash value and signature are internally encapsulated. Successfully validating an encrypted message requires two additional decryption operations—one with a private key and one with a session key—which is resource-intensive, primarily due to the use of an asymmetric algorithm.

3.2. First stage of encryption (symmetrical scheme)

$$E^{k^{\wedge}}(m) = (s \parallel EAEk'(m')),$$

$$k' = KDF(k^{\wedge}, s), m' = f(m), s = [RNG],$$

Where $k^{\wedge}$ – shared secret of subscribers, k' – session encryption key, s – cryptographic salt calculated for one message, m – open message, f – function of complementing the message to a constant value, KDF – key generation function, EAE – Authenticated encryption function.

This scheme allows to reduce the size of transmitted packets by reducing the overhead of adding header information in the form of key encapsulation procedures $E_{pubB}(k')$ and signing messages $SprivA(h)$. This property simplifies network operation in complex, message-size-sensitive environments, such as radio broadcasting and centralized communications services.

The security of this function depends directly on the quality of the entropy of the shared secret $k^{\wedge}$, which is then used to generate session keys k' , from the quality of the RNG / KSGPSN that

generates cryptographic salt s , and also from the security of the authenticated encryption functions themselves EAE_k and KDF key generation.

The KDF function, in combination with the cryptographic salt s , is primarily needed to protect against possible birthday paradox attacks targeting the initialization vector (IV/nonce) of the authenticated encryption function (primarily in the GCM encryption mode, where the nonce is 96-bit). When using the same encryption key and randomizing the nonce, the number of generated messages should not exceed $2^{\text{nonce-bit}/2}$ (for GCM, 232 according to the NIST recommendation [13]). When using a KDF with a random value s , this limitation is lifted.

Unlike a hybrid scheme, where node identification is determined through the use of public keys, in a symmetric encryption scheme, identification is replaced by node connectivity through the use of a shared key (secret). This changes the principle of message decryption from using a single private key to a list of available secrets.

3.3. Second stage of encryption

$$E''_{nk}(m) = \text{Enk}(p(h) || h || m),$$

$$h = H_{MAC(nk)}(m),$$

Where nk is the network key, p is the proof-of-work function, h is the hash result, and m is the plaintext message. The function follows the MtE (MAC-then-Encrypt) approach, where the MAC (Message Authentication Code) is first calculated, and then the message m , the resulting code h , and the proof $p(h)$ are encrypted using the Enk function.

This encryption stage serves one purpose: to differentiate between different networks using the network key nk , preventing them from being merged into a single system. This is achieved primarily through the proof-of-work function p , as it makes it more expensive to re-encrypt all traffic sent from one network with key k_1 to another network with key k_2 .

The proof-of-work function p is defined by the proof-of-work (PoW) algorithm [14], where for a specific hash value h , it is necessary to find a number i such that the result $h_i = H(h || i)$ is a bit vector with a certain n -th number of zeros as a prefix, for example 00000000(n)...11001010. The number n is called the complexity of the work.

It's also worth noting that the encryption key nk can be a public parameter if there's no need to form a specific group of nodes with a shared secret. In this case, the network key nk simply becomes a well-known setting for network delimitation.

The Encrypt-then-MAC (EtM) approach is not used in the second-stage encryption scheme for three reasons:

1. A single key nk is used for encryption and authentication, instead of two keys k_1 and k_2 for these tasks. If the EtM approach is applied in this situation, then the same key nk would be exposed to two attack vectors instead of one. This problem could be solved by using a KDF (key derivation function), which would allow multiple keys to be generated from a single key. However, this would complicate the overall encryption scheme and also open an additional attack vector against the KDF itself.

2. Horton's principle is taken into account: "authenticate not what is said, but what is meant" [6, p. 130]. If the MAC calculation function in the EtM approach is successfully attacked, or if the keys k_1 and k_2 are incorrectly distributed, a situation may arise where authentication will yield a

positive result for the encrypted message, but the decryption procedure itself will be incorrect. If the message is a chaotic set of bits, then we will never know its truth.,

3. The MtE approach hides the proof-of-work in the ciphertext, making the encryption result indistinguishable from a random bitstream. This property proves beneficial in situations where anonymized traffic can be blocked based on a generated pattern.

The MtE approach certainly has the drawback of requiring decryption before verifying the integrity and authenticity of the information. The EtM approach does not have this problem. This led to the development of Marlinspike's principle of cryptographic doom, which states: "If you are forced to perform any cryptographic operation before verifying the received message's cryptanalysis, then it will inevitably lead to a fatal outcome" [15, p. 93]. This principle is contrasted with Horton's principle when it comes to choosing between the two approaches: MtE or EtM. However, the choice of MtE is also dictated by the fact that the principle of cryptographic doom is also violated at the first, and much more expensive, stage of encryption.

4. Networking

The existence of a QB problem assumes that every message sent in the network will reach all its participants. However, such a problem says nothing about how the message will reach nodes in the absence of all-to-all communication, what data transfer protocol will be used, how applications should operate in such a model, how the decentralized structure will traverse NAT on the internet, etc. All these issues require further clarification based on the specific implementation of network interaction.

4.1 Microservice Architecture

Hidden Lake Anonymous Network as an App¹⁰, is a set of services, each of which performs its own specific task [16]. The choice of microservice architecture, as opposed to monolithic, was made taking into account the following aspects:

1. Microservice architecture allows for the simplification and decentralization of application service development, making it possible to use various programming languages and technologies when implementing your own applications.,

2. Microservice architecture allows for the separation of services' responsibilities for processed or stored information, so that as the system becomes more complex, the correlation of this complexity will be minimally distributed among services.

3. Microservice architecture allows you to add new features, edit them, or remove them completely without having to recompile and restart all services, which can have a positive impact on both testing and fault tolerance.

¹⁰Hidden Lake Anonymous Network Repository:<https://github.com/number571/hidden-lake>.

The core of the Hidden Lake network is the HLK service (kernel), which directly executes the QB task and all encryption/decryption functions, respectively. In addition to the HLK application, the Hidden Lake project also includes a number of application services and adapters. As a result, the Hidden Lake network can be represented as a composition¹¹ several services.

$$HLK = D + QB-net [E(nk, \langle fk \rangle)(m) = E''nk(E' \langle fk \rangle(m))] + HLA=http,$$

$$Hidden\ Lake = (\prod_{i=1} HLS_i) \times HLK \times (HLA=http)u(m) \times (\prod_{j=1} HLA_j),$$

where is *HLS*- a set of application services, *HLA* - a set of network adapters, *D* - a provider of open messages to application services, *HLA=http* - a network adapter based on the HTTP protocol, $u(m) = \lfloor (m-1)/(m+2) \rfloor$, which is equivalent to the expression: (1 if $m > 1$ else 0), for $m \in \{0\} \cup \mathbb{N}$. The *HLA=http* adapter has a dual nature: 1) an embedded (internal) application; 2) a separate (external) application. In the first case, *HLA=http* is part of a basic application (e.g., HLK or *HLA=<proto>*), whose function is to establish communication between multiple services using the HTTP protocol. In the second case, *HLA=http* is a separate application (service), whose function is to link (generalize) multiple network adapters under a single common interface.

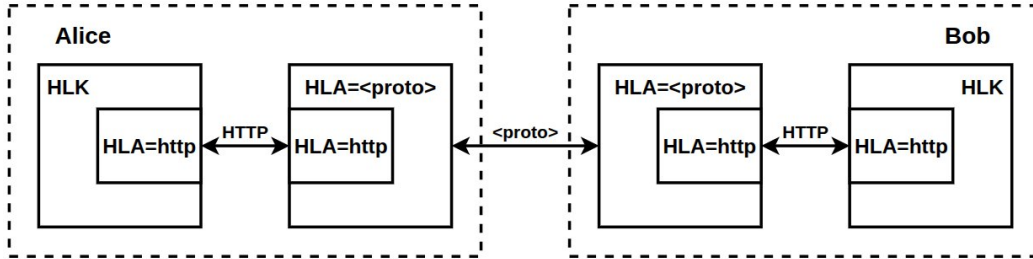


Figure 3. Interaction between two HLK nodes via *HLA=http* and *HLA=<proto>*

Without applications or network adapters (except *HLA=http*), a Hidden Lake network becomes equivalent to an HLK service: Hidden-Lake = HLK. This is the minimum characteristic required to remain a Hidden Lake network. If the HLK service is removed, the system ceases to be a Hidden Lake network, as it loses its core, even if application services and network adapters still exist.

Algorithm 3. Handling queries in an HLK application in pseudocode

INPUT: request req, sender s, service mapping M, filter function H
OUTPUT: rsp response OR algorithm completion
host, method, path, head, body ← req (Receiving a request *)*
*if (host ∉ M) {(*Service not found in display*)*
 return (End of algorithm *)*
}

¹¹ Symbol "+" (addition) defines the composition of functions within a single application, while the symbol "×" (multiplication) establishes the composition of functions through the use of multiple applications. The symbol "P" combines applications with similar operating logic into one common compositional group. $m = 0 \rightarrow u(m) = 0, HLA=http$ becomes equal to one, which leads to the following equality: $Hidden-Lake = \prod_{i=1} HLS_i = <app_i> \times HLK \times 1 = \prod_{i=1} HLS_i = <app_i> \times HLK$.

```

intReq ← M(host), method, path, (head || s), body(*Query enrichment*)
intRsp ← do(intReq) (* Request to the service *)
if (noResponse(intRsp)) {(* No response received OR No response required *)
    return(* Completion of the algorithm *)
}
status, head, body ← intRsp(*Getting a response*)
rsp ← status, H(head), body (*Response filtering*)
return rsp (* Sending response *)

```

As a result of all the above, the implementation of the Hidden Lake network can represent three different communication modes: classic, adaptive, and multiplicative.

1. Classic communication mode refers to connections based on a standard HTTP adapter implemented directly in the HLK application. This mode assumes that nodes or relays (based on HLA=http) pre-establish a list of possible connections so that they can not only receive but also successfully send messages, thereby establishing a full-duplex connection. It is recommended for use in situations where the use of the HTTP protocol is sufficient, and the network addresses of the participants are accessible, static, and known in advance (included in the connection list).

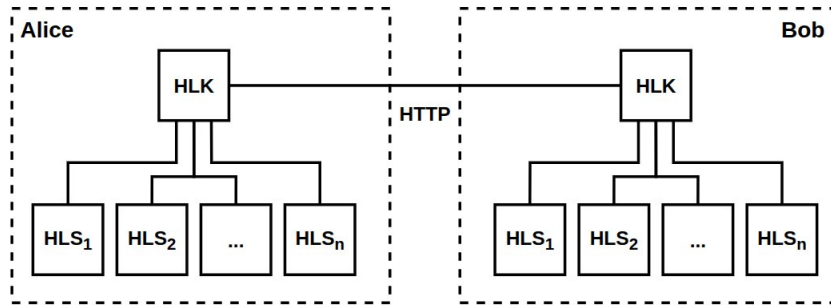


Figure 4.Classic mode:
 $Hidden-Lake = (\prod_{i=1}^n HLS_i) \times HLK$

2. An adaptive communication mode refers to connections based on a protocol (or communication platform) other than the standard HTTP adapter. This communication mode allows the implementation to be decoupled from the specific technology used (HTTP protocol) and then adapted to a variety of other protocols (TCP, UDP, QUIC, etc.) or communication platforms (social networks, instant messengers, forums, etc.). It is recommended for use in situations where the communication system has a number of limitations, such as the inability to set static network addresses, the list of network participants cannot be known in advance, the use of the basic HTTP protocol is impossible in the chosen communication environment, etc.,

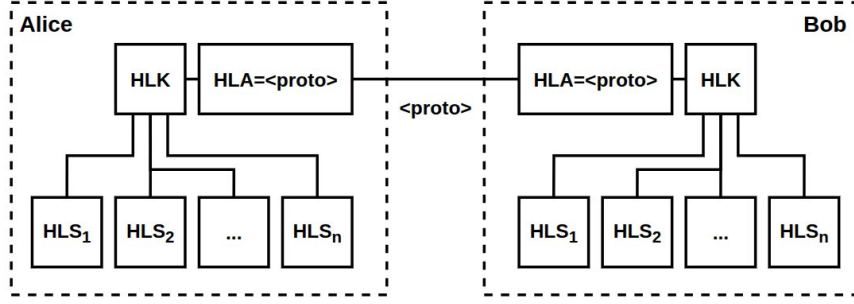


Figure 5.Adaptation mode:

$$Hidden-Lake = (\prod_{i=1}^n HLS_i) \times HLK \times HLA=<proto>$$

3. Multiplicative communication mode refers to connections based on multiple protocols and/or communication platforms connected by a single HTTP adapter. The standard HTTP adapter found in the HLK application is incapable of relaying messages, i.e., it is incapable of forwarding messages received from the $HLA=<proto_x>$ adapter to multiple other $HLA=<proto_i>$ adapters for $i=1, 2, \dots, x-1, x+1, \dots, m$. For this reason, a complementary HTTP adapter is required that adheres to both the basic protocol and relaying functions. It is recommended for use in situations where multiple communication platforms exist, most of which are unstable (weak hardware, incorrect operation, frequent API updates) or hostile to automation (requiring authorization, account blocking, setting timeouts between operations, checking task completion (solving captchas, executing JS scripts).

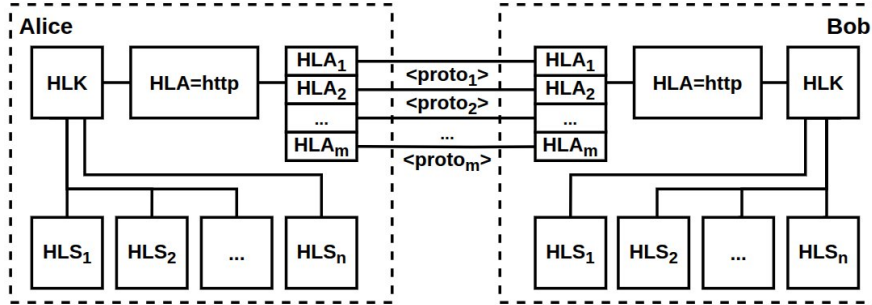


Figure 6.Multiplicative mode:

$$Hidden-Lake = (\prod_{i=1}^n HLS_i) \times HLK \times HLA=http \times (\prod_{j=1}^m HLA_j)$$

4.2. GP/12 Protocol Stack

The abstract nature of the Hidden Lake anonymous network is formed by the GP/12 protocol stack (short for go-peer¹² and 1,2 – encryption layers¹³), similar in essence to the TCP/IP protocol

¹²Go-peer project repository: <https://github.com/number571/go-peer> Go-peer contains all the core modules: encryption functions, message queuing, and networking. Hidden Lake, in turn, actively uses these components to implement services such as HLK and $HLA=tcp$. For the Hidden Lake network, the go-peer project previously served as a framework due to their shared codebase and release versions. These projects are now separate.

¹³Encryption layers are ordered in reverse order relative to the encryption stages, i.e., first encryption layer = second encryption stage, second encryption layer = first encryption stage. This difference in order stems from the different schemes on which these definitions are based. Encryption layers are based on how the message is received, and encryption stages are based on how the message is sent. To avoid confusion, this paper primarily uses the term "encryption stages," with the exception of the

stack. It also has four layers: data link (CL), network (NL), transport (TL), and application (AL), but has the following differences:

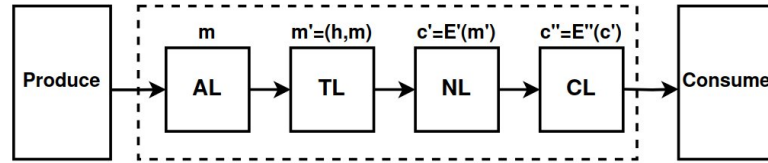


Figure 7.GP/12 protocol stack

1. GP/12 identifies nodes by public keys rather than IP addresses,
2. GP/12 can use the problem of theoretically provable anonymity,
3. GP/12 does not depend on network protocols and communication systems,
4. GP/12 uses an end-to-end encryption scheme.

The data link layer in the GP/12 network model distributes messages using flood routing. This layer is characterized by the use of a second-stage encryption when the successful transmission of a message to all connections is important. The network layer distributes messages to specific network nodes using public keys as identifiers. This layer is characterized by the use of a first-stage encryption when the confidentiality and authenticity of messages are important. The transport layer routes clear messages (successfully decrypted) to specific application services. The application layer receives clear messages and then processes them. Thus, the GP/12 protocol stack can be represented as a composition of four layers from the message reception perspective: $CL \times NL \times TL \times AL$.

Due to the above, the implementation of the full GP/12 protocol stack from the channel to the application layers is only possible using at least two services in the Hidden Lake network, since HLK, being the core of the network, covers only the first three layers: CL, NL, TL, while the last layer AL can only be covered by using application applications such as: HLS=messenger, HLS=files harer, HLS=remoter, HLS=pinger, etc.

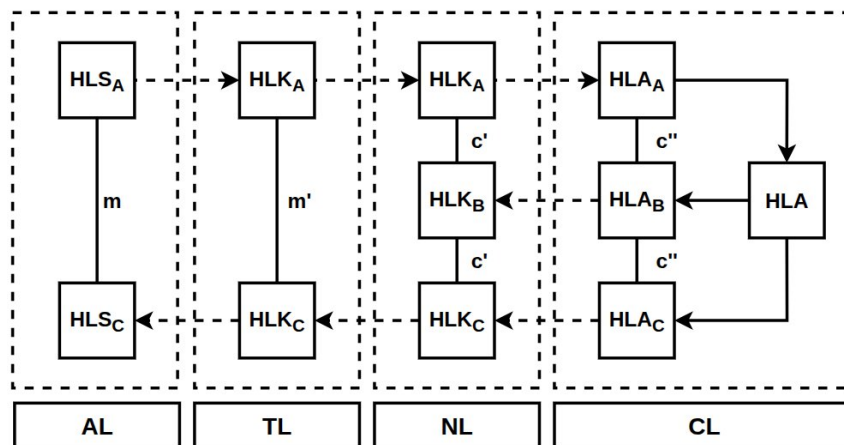


Figure 8.GP/12 model using the Hidden Lake network as an example, where A is the sender, C is the receiver

Also, the GP/12 network model does not necessarily have to have the anonymity property, which is why the HLK service, in the case of a remote QB task in order to maintain GP/12 compatibility, can be replaced by a composition of three services: $D \times \text{Encryptor} \times \text{HLA} = \langle \text{proto} \rangle$, providing the third, second, and first levels, respectively. The secpy-chat application was created based on this approach.¹⁴

Traffic anonymization is formed at the second level of the GP/12 stack, when the message generation function becomes inextricably linked to an anonymizing task. This could be a QB, DC, or EI task. The main limitation in this choice is the need for theoretical provability, without which it will be impossible to further develop an abstract system with the property of anonymity. If Proxy or Onion tasks are used, a network based on the GP/12 network model will not become more anonymous, because these tasks, for their correct execution, require a non-monolithic system, which contradicts the definition of abstract anonymity networks, for which the presence of centralization does not affect the quality of the resulting anonymization.

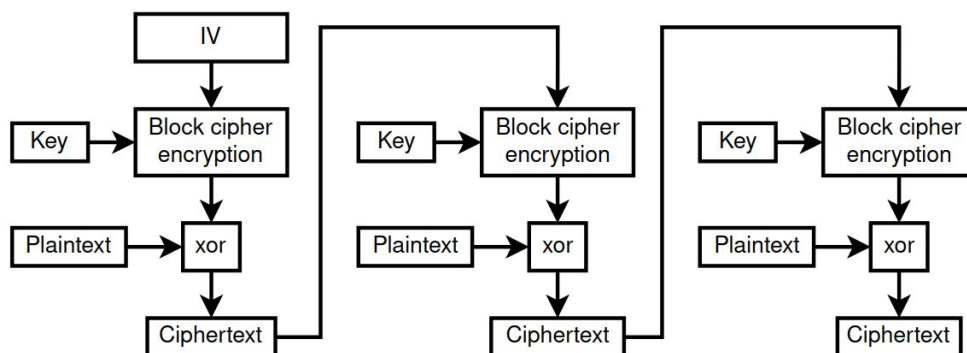
The GP/12 protocol stack requires only the existence of the Produce and Consume network interfaces to implement its implementation, allowing it to adapt further to a specific communication environment. For example, the GP/12 model in the Hidden Lake network is based on the HTTP application-layer protocol by default, and with the help of HLA services (adapters), it can be built on top of transport-layer protocols such as TCP, UDP, and QUIC; application-layer protocols such as SSH, FTP, and SMTP; or even beyond the TCP/IP model entirely, such as using radio broadcasting or visible light.

5. Software implementation

Mathematical models allow us to determine the correctness of the overall operating logic, but they do not allow us to determine the security of a specific implementation. For example, we can only assume that a function or the value it accepts will be secure. However, this does not reveal anything about the specific implementation, the selected parameters, the threat model, or the design approaches. Therefore, it is necessary to focus not only on a general description of the Hidden Lake network's operation but also on a detailed description of its structural and configuration parameters.

5.1 Structural parameters

1. The symmetric encryption function E_k is defined by the AES block algorithm with a key length of 256 bits in the CFB encryption mode, where $c_0 = IV$, $c_i = E_k(c_{i-1}) \text{ xor } m_i$.



¹⁴secpy-chat application repository: <https://github.com/number571/secpy-chat>.

Figure 9.CFB encryption mode(ciphertext feedback mode)

This encryption mode was chosen taking into account the following points:

1. CFB encryption mode does not require padding, as is required, for example, by CBC (cipher block chaining). This simplifies setting a static encrypted message size and eliminates potential oracle attacks.[17],
 2. CFB encryption mode is not a stream-based encryption mode, such as OFB (output feedback mode) or CTR (counter mode). Therefore, CFB does not suffer from the problem of gamma repeatability. If the IV (initialization vector) is repeated, this will lead to far fewer security issues than with OFB or CTR modes [6, p. 93].
 3. CFB encryption mode does not require encryption algorithms to adhere to precise parameters, such as 128-bit blocks in GCM mode.(counter mode with Galois authentication)[18, p.190], and also does not require decryption functions from the algorithms, which allows not only to simplify the replacement of insecure ciphers, but also to use one-way functions as an encryption procedure [19, p.282],
 4. GCM encryption mode was also not used due to the unnecessary authentication and token storage operations. The first (hybrid) and second stages of encryption use different message authentication methods. Therefore, CFB mode is preferable in terms of flexibility.
2. The symmetric encryption function with authentication EAEk, used in the symmetric first-stage encryption scheme, is defined by the AES block algorithm with a key length of 256 bits in the GCM encryption mode., where $c_i = E_k(\text{nonce} \parallel i+1) \text{ xor } m_i$, $T = \text{GHASH}(A, C) \text{ xor } E_k(0128)$. Unlike the second stage, the encryption key used does not define the network group of participants, but the connection between the direct subscribers of the communication,
3. The asymmetric encryption function Epub is defined by the ML-KEM-768 algorithm (before standardization – Kyber-768) [20]. The asymmetric signing function Spriv is defined by the ML-DSA-65 algorithm (before standardization – Dilithium-M3) [21]. The choice of such algorithms was made taking into account the preparation of the Hidden Lake network for post-quantum cryptography. The establishment of an F2F connection to a communication subscriber is actually determined by two public keys – ML-KEM-768 for encrypting Epub messages (recipient key) and ML-DSA-65 for verifying the signature of Vpub messages (sender key).
4. The network key in the second stage of encryption is assumed to change infrequently, have high entropy, and not be a password. However, since the network key is not fixed in size, it is passed through the PBKDF2 key derivation function to be fixed at 32 bytes, as required by the AES-256 encryption algorithm. The salt and iteration count are not specified in PBKDF2, meaning they default to the empty string and the number 0. The hash function is SHA-512.,
5. HMAC-SHA-384 is used as the hash insertion algorithm (MAC). The security of HMAC depends on the hash function it uses [18, p. 168]. The security of SHA-384 can be determined by its 384-bit strength for the first and second preimage search problem, and by 192-bit strength for collision search, based on birthday paradox attacks [6, p. 52].,

6. Quantitative and immutable parameters of the AES-256, SHA-384, ML-KEM-768, ML-DSA-64 algorithms were chosen from a conservative point of view to maintain a sufficient level of security in the realities of post-quantum cryptography [5, p. 131]. Grover's algorithm theoretically makes it possible to reduce the security of symmetric ciphers to brute-force attacks and hash functions to collision searches by reducing the required number of calculations to $2^{n/2}$ and $2^{m/3}$, where n is the length of the symmetric cipher key, m is the size of the resulting hash function block. Thus, when using AES-256 and SHA-384, the minimum security will be determined by 128 bits, which is a value resistant to brute-force attacks. In the absence of quantum computers, the minimum security will be determined by the ML-KEM-768 and ML-DSA-64 algorithms, the security of which is comparable to a 192-bit value,

7. Neither the first nor the second stage of encryption protects against a replay attack [6, p.279], where after a certain period of time t an attacker could retransmit a previously stored encrypted message. This message would be completely valid, as it was generated by one of the network participants. To protect against this type of attack, the Hidden Lake network adopts the simplest and most radical approach, storing message hashes in its local database. This approach completely eliminates the possibility of successfully accepting duplicates, including during node reboots, and also simplifies the overall deduplication process by eliminating additional message checks within time windows.

Unfortunately, protecting a node from receiving and subsequently processing duplicate messages does not protect against an attacker replaying messages that have not yet been received by the node directly owning the database. This can occur if the node was absent during the periods when ciphertexts were generated by the remaining system participants. Consequently, an attacker can overload a newly arrived node with old and long-forgotten network messages. This attack can be protected by connecting to a long-running anonymizing node or relay that has retained a history of received messages. In this case, it will act as a firewall, separating old and new ciphertexts. Another possible solution is to update the network key k if the system has generated a lot of traffic and long-running nodes are unknown or unavailable.

Adding a timestamp t to each message in the second stage of encryption, on the other hand, can compromise the anonymity of the QB problem, since if implemented incorrectly, it can create an additional link in ciphertext generation and make the systematicity problem $P(s)$ easily solvable. For example, when generating a genuine message, the timestamp will be fixed not at the moment of sending, but at the moment of generation. Because messages from the queue are sent sequentially to the network, the pre-generated false ciphertext will have a timestamp close to that of the plaintext message. In other words, the generation of genuine messages will be linked by the existence of two timestamps, the difference between which is less than one period. This situation can be corrected by replacing the false ciphertext with the true one, rather than simply pushing the false ciphertext back in the queue. But, in addition to this problem, the ciphertext generation mechanism must also take into account the timing of the timestamp t , depending on the generation period T . In other words, for the n th message in the queue, the timestamp should be as follows: $t = \text{now} + nT$, where now is the current time. This solution can also have a negative effect if the ciphertext generation rate, for example, under heavy load on the node, does not keep up with the timestamp. In this case, the queues may become desynchronized by the timestamp between genuine and false messages. This situation can be corrected by constantly removing old generated ciphertexts from the queue, but this runs the risk of losing legitimate messages that were never even sent to the network.

8. In some situations, the second stage of encryption can be omitted at the adapter level if the system in which Hidden Lake is deployed already uses group-based authentication by default. This is typically done in conjunction with a symmetric encryption scheme in the first stage when the platform or communication protocol requires the smallest possible packet size, such as 200 bytes in the Meshtastic/LoRa radio environment.,

9. Unlike the classical description of QB networks, in the go-peer implementation, the queue structure Q is represented by two queues: a queue of true Q_k messages and a queue of false Q_r messages, which are then merged into one. The need for two queues is driven by the need for background generation of false messages, so that the Q_r queue is predominantly non-empty when the period t is reached. This need, in turn, is directly related to the proof-of-work algorithm, which significantly slows down message encryption, as a result of which the set generation period t can be extended. Although the situation in this case is similar to Example T1:3, the systematicity of the algorithm is nevertheless not violated here, since the generation of true and false messages occurs not at the moment of their sending, but at the moment they are placed in the queue.

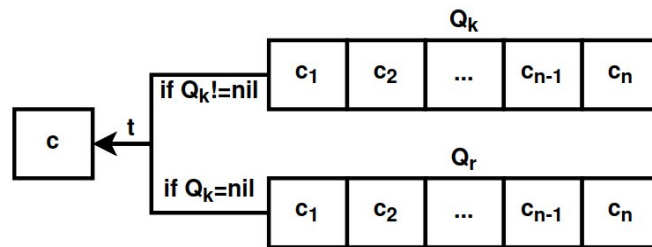


Figure 10. Double message queue diagram in the go-peer project

10. The Hidden Lake anonymous network may be vulnerable to active cooperating observers if they are present in the friend list of several subscribers in a system without multi-queues. In this case, in addition to degrading anonymity quality (from the task of concealing the fact of communication to concealing the connection between subscribers), there is also a risk of solving/deanonymizing the QB problem itself. This problem can be solved by creating multiple queues Q_{k1} , Q_{k2} , ..., Q_{kn} for each individual subscriber (using the `qbp_consumers=n` setting), but this will increase the overall response time by a factor of n . Another solution to this problem could be a change to the threat model, in which a friend is a priori considered a trusted node and therefore will not be interested in or involved in deanonymization procedures. The Hidden Lake network is primarily based on this F2F system design.

11. MThe threat model for the Hidden Lake anonymous network is limited solely to protecting network communications between its participants, which, in turn, presupposes the absence of any additional measures, actions, or conditions aimed at protecting private keys, databases, configuration files, or service interactions within the local execution environment. Consequently, measures taken to ensure the security of the local environment, especially when sensitive information exists, should be delegated to lower levels of interaction, such as setting file and process access rights, isolating programs using a virtual environment, software and hardware full-disk encryption, restricting physical access to hardware for third parties, etc.

5.2 Configuration parameters

1. The approach of storing hashes of all previously received messages also has a drawback: the database's size tends to constantly grow. Once a hash database is created, it must never be deleted or cleared, otherwise all previously received messages could be repeated if an attacker uses a replay attack. In this paradigm, the issue becomes based on the volume and frequency of continuously generated information. A message hash in the Hidden Lake network is the hash value $h = H(c')$ obtained from the ciphertext in the first stage of encryption: $c' = E'/k$. The hash size is determined by the SHA-384 cryptographic hash function, i.e., 48 bytes. Therefore, one received or sent unique message will increase the database by 48 bytes. The message generation period for one node is 5 seconds. Assuming there are 10 nodes in the network, each generating one encrypted message every 5 seconds, then 480 bytes will be generated and stored in 5 seconds, or 96 bytes per second. Further, if we extend this result to a year, the database will increase by ~ 2.781 GB.¹⁵ information, which is a completely acceptable value,

2. With each stored hash in the database, the total space of valid ciphertexts decreases due to the associated collisions inherent in all hash functions. Consequently, the more ciphertexts are accepted, the fewer they can be accepted in the future. The probability of finding a collision in more than 50% of cases begins after exceeding the threshold of 2192 accepted ciphertexts for the SHA-384 hash function using classical computers. After this, the network will become unstable, discarding half of all messages received by the system, and the only way to "reset" the network will be to delete the database and then change the network key and all private keys. However, such an event is impossible, as it requires storing $249230249209671726169463823802564608$ YiB¹⁶ information for 2192 SHA-384 hashes. If there are 8000 nodes in the network with a ciphertext generation pattern of 1.6 KiB/s, i.e., when the communication channel limit of 100 Mbps is reached, ~ 2224.8 GiB will be stored per year.¹⁷ in a local database. At this rate, it will take more than 1047 years to reach the collision detection threshold of $>50\%$ of cases.¹⁸

3. Messages at the first stage of encryption have a static size of 8192 bytes, of which 4569 bytes are allocated to header data: initialization vector (16B), data hash (48B), ML-DSA-65 signature (3309B), sender's public key hash (48B), salt (32B), encapsulated ML-KEM-768 encryption key (1088B), as well as the data sizes in bytes: hash (4B), signature (4B), public key (4B), salt (4B), ciphertext block (4B), payload (4B), padding bytes (4B). The second stage of encryption adds 76 bytes, of which: initialization vector (16B), proof of work (8B), ciphertext hash (48B), network mask (4B). The message size after full encryption is 8268 bytes, of which 3623 bytes are payload. Therefore, if the message generation period is 5 seconds, then a node can transmit ~ 724.6 significant bytes per second. If the application assumes request-response communication, then, due to the request made and the existing sequence (queue), a response waiting period will inevitably begin, which will lead to a twofold decrease in throughput to ~ 362.3 significant bytes per second due to the increased waiting interval of ~ 10 seconds. Therefore, if there is a task to transfer

¹⁵ $((48 \text{ [bytes]} \times 10 \text{ [knots]} / 5 \text{ [seconds]}) \times 60 \text{ [seconds]} \times 60 \text{ [minutes]} \times 24 \text{ [hours]} \times (7 \text{ [days]} \times 4 \text{ [weeks]} + 2 \text{ [days]}) \times 12 \text{ [months]}) / 230 \text{ [GiB]} = 2.780914306640625 \approx 2.781 \text{ Gigabytes.}$

¹⁶ $(2192 \text{ [hashes]} \times 48 \text{ [bytes]} / 280 \text{ [YiB]}) = 249230249209671726169463823802564608 \text{ YiB.}$

¹⁷ $(8000 \text{ [nodes]} \times 2.781 \text{ [GiB} \times 10 \text{ knots]}) / (10 \text{ [knots]}) = 2224.8 \text{ [GiB} \times 8000 \text{ nodes].}$

¹⁸ $(2192 \text{ [hashes]} \times 48 \text{ [bytes]}) / ((48 \text{ [bytes]} \times 8000 \text{ [nodes]} / 5 \text{ [seconds]}) \times 60 \text{ [seconds]} \times 60 \text{ [minutes]} \times 24 \text{ [hours]} \times (7 \text{ [days]} \times 4 \text{ [weeks]} + 2 \text{ [days]}) \times 12 \text{ [months]}) = 126131320235875626202333088654346434865739845447 > 1047 \text{ years.}$

a 1 MiB file over the network, then transportation will take approximately ~24.1 to ~48.2 minutes (~1447 and ~2894 seconds, respectively).

4. The Hidden Lake network's complexity is determined by the `work_size_bits` parameter, which in the standard (recommended) configuration is 22 bits. This parameter limits the throughput of each network node, thereby mitigating DoS and DDoS attacks, as well as attacks aimed at duplicating messages across multiple subnets. This parameter can determine the approximate numerical limit of network participants.

Let's assume that the throughput of each node in the network is 100 Mbps, the generated message size is 8 KB, and the generation period is 5 seconds. In this case, network congestion would be possible if there were more than 8,000 nodes.¹⁹ This, in turn, is an upper bound that does not take into account the possibility of malicious actions by the nodes themselves, capable of generating traffic at more than 8 KiB/5 sec = 1.6 KiB/sec. Without taking into account the `work_size_bits` parameter (the second stage of encryption), the natural limitation for attackers becomes the message encryption time (the first stage of encryption). On an Intel(R) Core(TM) i7-9750H CPU @ 2.60 GHz processor, encryption of one 8 KiB message takes approximately 0.254 ms, which requires an average of 0.4 seconds.²⁰ to reach the maximum value of 12.5 MiB, with the further possibility of overflowing the network channel at 250 Mbit/s = (12.5/0.4)×8. As a result, 8000 honest nodes become equal to less than one malicious one≈0.4.

To prevent this discrepancy, a parameter is introduced that determines the work complexity. With `work_size_bits`=22, encrypting one 8KiB message takes approximately 1.98 seconds on the same processor, excluding the first encryption stage, which averages 52.8 minutes.²¹ (3168 seconds) to reach the limit of 12.5 MiB. As a result, to reach the limit, an attacker would need ~3168 parallel calculations of the same processor power to be able to overcome the channel bandwidth of 100 Mbps. Continuous encryption of messages with `work_size_bits`=22 consumes ~15% of the processor power, which allows for an increase in parallelism by ~6.(6) times to reach the limit of 100%. Thus, ~3168 parallel calculations are comparable to using 3168 / 6.(6)≈475.2 physical processors operating at their capacity limits.

The attacker's situation becomes easier if a GPU is used instead of a CPU. The SHA-384 hashrate for the "Nvidia GTX 1080 Founders Edition" is ~1050 MH/s.²², while the hashrate of "Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz" is ~26.6 MH/s²³ As a result, the difference between the processors is determined to be 1050 / 26.6≈39.5 units, which is equivalent to a ratio of 475.2 CPU to 475.2 / 39.5≈ 12 GPUs when solving a PoW problem.

6. Conclusion

This paper analyzed the operation of the Hidden Lake anonymity network based on its mathematical models, then outlined the advantages and limitations arising from the QB problem.

¹⁹ $(100[\text{Mbit/s}] / 8[\text{bits}] = 12.5[\text{MiB/s}]) / ((8[\text{KiB}]/5[\text{s}] = 1.6[\text{KiB/s}]) / 1024[\text{bytes}] = 0.0015625[\text{MiB/s}]) \approx 8000$ nodes, each generating 1.6 KiB/s.

²⁰ $(8[\text{KiB}] / 0.254[\text{ms}]) \rightarrow (1.6[\text{KiB}] / 0.0508[\text{ms}]), 0.0508[\text{ms}] \times 8000[\text{packets of 1.6 KiB}] = 406.4 \text{ ms} \approx 0.4 \text{ s}.$

²¹ $(8[\text{KiB}] / 1.98[\text{s}]) \rightarrow (1.6[\text{KiB}] / 0.396[\text{s}]), 0.396[\text{s}] \times 8000 [\text{packets of 1.6 KiB}] = 3168 \text{ s} = 52.8 \text{ m}.$

²² "8x Nvidia GTX 1080 Hashcat Benchmarks":

<https://gist.github.com/epixoip/a83d38f412b4737e99bbef804a270c40>.

²³ At 15% CPU utilization, the SHA-384 hashrate is ~4 MH/s. At 100% CPU utilization, the hashrate increases by a factor of 6, reaching 26.6 MH/s.

The network's main services were described, along with their network interactions within a microservice architecture paradigm. The GP/12 protocol stack, which enables networks with theoretically provable anonymity to become abstract, was described. Criteria for selecting encryption, signing, and hashing algorithms, along with their parameters and settings, were presented during network design and implementation.

6.1. Applied application

The Hidden Lake anonymity network, while possessing all the advantages and disadvantages described above, also defines the boundaries of its application. Unlike the Tor and I2P networks, whose scope of application is quite broad, both due to the lack of interconnectivity between communication subscribers and a reduced threat model (the absence of a global observer), the Hidden Lake network, on the contrary, is severely limited in its use cases, as low communication channel bandwidth (generation periods) and scalability issues (blind routing) prevent HL from becoming a truly monolithic system. Consequently, the only way to expand the network is horizontal scaling by creating multiple, isolated subnets. Despite all the shortcomings, limitations, and inconveniences of use in specific situations, the Hidden Lake network still has a potential list of useful applications:

1. Protecting local networks from eavesdropping. The scalability issue is most acute in global networks and least noticeable in local networks. Due to this, and the lack of any monolithic nature, the Hidden Lake network can be easily deployed and used in small communications systems. In turn, the probability of the presence of a global observer in local networks is incomparably higher than in global ones, and therefore the ability of an anonymous network to counteract global observations becomes one of the important conditions,

2. Protecting military networks from eavesdropping. The abstract nature of the Hidden Lake anonymous network allows it to adapt to specific communication conditions. In turn, the proper construction of friend-to-friend relationships (e.g., a hierarchical structure) helps reduce the risk of information compromise between multiple network participants. Traffic anonymization helps conceal the actual activity and communication structure of its subscribers. The existence of specific generation periods and knowledge of the exact number of nodes in the network helps prevent DoS and DDoS attacks if a particular part of the system begins generating more traffic than the limiting constant.

3. Formation of covert communication channels. Adapters in the Hidden Lake network allow not only adaptation to specific network protocols but also the integration (injection) of anonymizing traffic into existing centralized and/or decentralized systems. By increasing overhead by extending generation periods, reducing the bandwidth of the communication channel, and additionally using cryptographic or steganographic primitives, the generated traffic can be hidden not only from external third-party observers but also from the system itself.

This list of useful applications is not exhaustive, as it represents only those cases in which Hidden Lake excels compared to most other anonymity systems. Beyond this list, there are also overlapping areas. For example, Hidden Lake can be used quite effectively to perform specific functions and algorithms on the global network, such as remote access, website browsing, messaging, file sharing, and so on. The only limitations in this case are the number of participants

in the network, due to existing scalability issues, and the streaming communication, due to the ciphertext generation periods.

The Hidden Lake network threat model is constructed in such a way that observers can only be external and internal attackers, but not friends—that is, direct communication subscribers. This complicates and limits a number of applications in which communication subscribers must remain anonymous to each other. The use of repeaters/routers, in conjunction with channel or multiple encryption, makes it possible to complicate the analysis of network communications by separating the communications of subscribers from each other. However, this approach does not solve the problem thoroughly, since in cooperation with a passive or active global observer (depending on the measures taken), one of the communication subscribers still has the opportunity to deanonymize.²⁴your interlocutor.

Bibliography

1. Popescu, B., Crispo, B., Tanenbaum, A. Safe and Private Data Sharing with Turtle: Friends Team-Up and Beat the System [Electronic resource]. — Access mode: https://web.archive.org/web/20070316085325/http://www.turtle4privacy.org/documents/sec_prot04.pdf (date accessed: 04.07.2024).
2. Kovalenko, G. General Theory of Anonymous Communications. Second edition / G. Kovalenko.— [b. m.]: Publishing Solutions, 2023. - 208 p..
3. Schneier, B. Applied Cryptography. Protocols, Algorithms, and Source Codes in C / B. Schneier. - St. Petersburg: Alfa-Kniga LLC, 2018. - 1040 p..
4. Alferov, A., Zubov, A., Kuzmin, A., Cheremushkin, A. Fundamentals of Cryptography: A Tutorial / A. Alferov, A. Zubov, A. Kuzmin, A. Cheremushkin. - M.: Helios APB, 2002. - 480 p.
5. Grimes, R. The Cryptographic Apocalypse / R. Grimes. - M.: DMK Press, 2020. - 290 p.
6. Schneier, B., Ferguson, N. Practical Cryptography / B. Schneier, N. Ferguson. - M.: Williams Publishing House, 2005. - 420 p.
7. Tanenbaum, E., Weatherall, D. Computer networks / E. Tanenbaum, D. Weatherall. - St. Petersburg: Piter, 2017. - 960 p.
8. Reiter, M., Rubin, A. Crowds: Anonymity for Web Transactions [Electronic resource]. — Access mode: https://www.cs.utexas.edu/~shmat/courses/cs395t_fall04/crowds.pdf (date accessed: July 4, 2024).
9. Perry, M. Securing the Tor Network [Electronic resource]. — Access mode: <https://www.blackhat.com/presentations/bh-usa-07/Perry/Whitepaper/bh-usa-07-perry-WP.pdf> (date accessed: July 4, 2024).
10. Astolfi, F., Kroese, J., Oorschot, J. I2P - Invisible Internet Project [Electronic resource]. — Access mode: https://staas.home.xs4all.nl/t/swtr/documents/wt2015_i2p.pdf (date accessed: July 4, 2024).
11. Danezis, G., Dingledine, R., Mathewson, N. Mixminion: Design of a Type III Anonymous Remailer Protocol [Electronic resource]. — Access

²⁴ In this case, deanonymization means finding the subscriber's network address, but not his actual activity with other network participants. In the QB problem concept, this cannot be called deanonymization, since the role of relays or routers is by definition absent. This means that the QB problem does not attempt to hide communication subscribers from each other, and all measures taken to anonymize subscriber connectivity begin to be reduced to other tasks.

- mode:<https://web.archive.org/web/20170312061708/https://gnunet.org/sites/default/files/minion-design.pdf>(date accessed:July 4, 2024).
12. Ishkuvatov, S. Method and algorithm for determining the type of traffic in an encrypted communication channel [Electronic resource]. — Access mode:<https://cyberleninka.ru/article/n/sposob-i-algoritm-opredeleniya-tipa-trafika-v-shifrovannom-kanale-svyazi>(date accessed:July 4, 2024).
 13. Dworkin, M. Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC [Electronic resource]. — Access mode:<https://nvlpubs.nist.gov/nistpubs/legacy/sp/nistspecialpublication800-38d.pdf>(date accessed:May 25, 2026).
 14. Nakamoto, S. Bitcoin: A Peer-to-Peer Digital Cash System [Electronic resource]. — Access mode:https://bitcoin.org/files/bitcoin-paper/bitcoin_ru.pdf(date accessed:July 4, 2024).
 15. Khlebnikov, A. OpenSSL 3: The Key to the Secrets of Cryptography / A. Khlebnikov. - M.: DMK Press, 2023. - 300 p..
 16. Lewis, J., Fowler, M. Microservices [Electronic resource]. — Access mode:<https://martinfowler.com/articles/microservices.html>(date accessed:September 7, 2024).
 17. Heaton, R. The Padding Oracle Attack[Electronic resource]. — Access mode:<https://robertheaton.com/2013/07/29/padding-oracle-attack/>(date accessed:July 4, 2024).
 18. Aumasson, J. Taking Cryptography Seriously: A Practical Introduction to Modern Encryption / J. Aumasson.— M.: DMK Press, 2021. - 328 p..
 19. Mao, V. Modern cryptography: theory and practice / V. Mao. - M.: Williams Publishing House, 2005. - 768 p.
 20. Module-Lattice-Based Key-Encapsulation Mechanism Standard [Electronic resource]. — Mode:<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf> (date accessed:October 19, 2024).
 21. Module-Lattice-Based Digital Signature Standard [Electronic resource]. — Mode:<https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf> (date accessed:October 19, 2024).